DESIGN AND ANALYSIS OF ALGORITHMS

ASSIGNMENT 1

Design, Analyse and Validate an Efficient Search Strategy for a Large-Scale Student Record System

Name: P SAI DEEPAK

Registration Number: 192411047

Course: Design and Analysis of Algorithms (DAA)

CO: CO1

Report date: 16 October 2026

GitHub repository: Add the repository URL here after creating the GitHub repository.

This report contains all twelve tasks. Experimental values were measured by running the included Python benchmark in the generation environment. Re-run the benchmark on the submission computer and capture genuine terminal screenshots before final submission.

Contents

Task 1: Algorithm design

Task 2: Basic operation

Task 3: Mathematical complexity derivation

Task 4: Best, average and worst cases

Task 5: Asymptotic analysis

Task 6: Recurrence formulation

Task 7: Recurrence solution

Task 8: Implementation and testing

Task 9: Experimental performance

Task 10: Theory versus experiment

Task 11: Final design recommendation

Task 12: Day-4 design challenge

Appendix: Reproduction instructions and limitations

Task 1 – Design the Algorithm

The system must determine whether a registration number exists in a dataset that grows from 1,000 to 1,000,000 or more records. Three distinct strategies are compared: iterative linear search, recursive binary search, and hash-index lookup. Linear and binary search are mandatory in the submission rules. The third strategy is hash indexing; verify the instructor-assigned group because the rules give different third strategies for different roll-number groups.

Algorithm 1: Iterative linear search

Idea: Inspect each record from the beginning until the key is found or the sequence ends. No preprocessing is required, and the input need not be sorted.

```
LINEAR-SEARCH(A, key)
  for i = 0 to length(A)-1
    if A[i] == key
      return i
  return -1
```

Design strategy: Brute force / sequential inspection. It is included as a simple baseline and is useful for small or unsorted collections.

Algorithm 2: Recursive binary search

Idea: Search a sorted array by comparing the key with the middle element and recursively continue in only one half. Sorting is required unless the data is already maintained in sorted order.

```
BINARY-SEARCH(A, key, low, high)
  if low > high: return -1
  mid = floor((low + high) / 2)
  if A[mid] == key: return mid
  if key < A[mid]: return BINARY-SEARCH(A,key,low,mid-1)
  return BINARY-SEARCH(A,key,mid+1,high)
```

Design strategy: Divide and conquer. It is included because it reduces the remaining search space by about half per recursive call.

Algorithm 3: Hash-index lookup

Idea: Build a hash set containing all registration numbers. A lookup tests membership in the set and is expected O(1) under a suitable hash distribution. The index requires O(n) preprocessing time and O(n) extra space.

```
BUILD-INDEX(A)
  index = empty hash set
  for each record x in A: insert x into index
  return index
HASH-SEARCH(index, key)
  if key is in index: return key
  return NOT-FOUND
```

Design strategy: Indexing / hashing. It is included because many repeated queries can amortize the one-time index construction cost.

Task 2 – Identify the Basic Operation

Linear search: The key equality test A[i] == key is the basic operation. It occurs once per inspected record, so a missing key requires n comparisons and a key near the end requires almost n comparisons. It directly measures the amount of sequential work.

Recursive binary search: The comparison between the key and the middle element is the basic operation. Each recursive level discards roughly half the current range, so the number of levels grows logarithmically. The implementation may perform equality and ordering comparisons at each level; the recorded counter counts one visited midpoint per level.

Hash lookup: The key-membership operation in the hash set is the basic operation. Its expected number of probes is constant under normal hash-distribution assumptions, but collisions can make individual lookups slower. The benchmark does not claim that hash probes and array comparisons are directly equivalent.

Task 3 – Derive the Time-Complexity Function Mathematically

Linear search

Let n be the number of records. For a missing key, every record is checked: C(n) = Σ(i=1 to n) 1 = n. For a successful search at zero-based position k, C(n,k) = k + 1. Therefore the worst-case search time can be written T(n) = a·n + b, where a is the cost of one key comparison and b is fixed overhead. There is no preprocessing, so total cost remains Θ(n) in the worst case.

Recursive binary search

Assume the input is sorted. For n elements, one midpoint is examined and at most one subproblem of approximately n/2 elements remains. Thus T(n) = T(■n/2■) + c, for n > 1, with T(0) = d and T(1) = e. After j levels, the subproblem size is approximately n/2■. Stop when n/2■ ≤ 1, so j ≥ log■ n. Hence search work is Θ(log n). If sorting is required, comparison sorting costs Θ(n log n), so total one-off cost is Θ(n log n) + Θ(log n) = Θ(n log n). If data is already sorted or maintained in sorted order, that sorting cost may not be paid per query.

Hash-index lookup

Building the index inserts each of n records once. Under expected constant-time insertion, B(n) = Σ(i=1 to n) O(1) = O(n). Each lookup is expected O(1), so for q queries the expected total is O(n + q), with O(n) additional space. Worst-case hashing can degrade toward O(n) per query if many keys collide; expected performance assumes a reasonable hash distribution and implementation.

For this experiment, preprocessing and search are timed separately. The table records two lookups per dataset, one present key and one absent key. Timing differences at microsecond scale are sensitive to runtime and machine load.

Task 4 – Analyse Best, Average and Worst Cases

Linear search: Best case is the key at the first position, requiring one comparison: Θ(1). For a uniformly positioned successful key, the average comparison count is (n+1)/2. An absent key requires n comparisons, giving Θ(n) worst-case time. This is why the absent personal key (registration number + 1) is a useful worst-case test.

Recursive binary search: Best case occurs when the key is at the first midpoint, with one midpoint visit: Θ(1). Average and worst cases follow a path through the binary partition tree and use Θ(log n) midpoint visits. An absent key also takes logarithmic time because the interval shrinks until it becomes empty.

Hash lookup: Expected best and average lookup time are Θ(1), assuming a suitable hash distribution. Worst-case lookup can be Θ(n) if collisions force extensive probing or bucket traversal. Index construction is expected Θ(n), independent of whether a particular query is found.

Task 5 – Perform Asymptotic Analysis

Linear search has worst-case O(n), Ω(1) and Θ(n) for a full scan such as a missing-key search. Binary search has O(log n), Ω(1) and Θ(log n) in the worst/typical logarithmic search path, with a constant-time best case. Hash lookup has expected Θ(1) query time and expected Θ(n) index construction; its worst-case query bound can be O(n).

At n = 1,000,000, linear search may inspect one million records for an absent key. Binary search needs about log■(1,000,000) ≈ 19.93 halvings, so roughly 20 midpoint levels, with the exact count depending on the key and array length. Hash lookup avoids scanning the records on each query but stores an additional index. The measured recursive binary search counters include both found and absent cases and show 40 total midpoint visits at one million records, consistent with roughly 20 per query.

Task 6 – Formulate the Recurrence Relation

For recursive binary search, the original input size is n sorted records. Each call examines one midpoint, performs constant work c, and makes at most one recursive call on a subproblem of size at most ■n/2■. The base case occurs when low > high (empty interval) or when the key matches the midpoint.

```
T(n) = T(floor(n/2)) + c, for n > 1
T(0) = d
T(1) = e
```

There is one recursive call per nonterminal call, not two. The search interval is halved because the comparison determines which half can still contain the key.

Task 7 – Solve the Recurrence Relation

The iteration method applies because each call reduces the input size by approximately a factor of two and does constant work outside the recursive call.

```
T(n) = T(n/2) + c
     = T(n/4) + 2c
     = T(n/8) + 3c
     = T(n/2^k) + kc
```

The recursion stops when n/2■ ≤ 1, so 2■ ≥ n and k ≥ log■ n. At the stopping point the remaining base-case cost is constant. Therefore T(n) = c·■log■ n■ + O(1) = Θ(log n). The result means that doubling the sorted dataset adds about one more midpoint visit, rather than doubling the work.

Task 8 – Implement and Test the Algorithms

The implementation is in src/. The dataset generator uses seed 1047, the last four digits of registration number 192411047. It generates datasets of 1,000, 10,000, 100,000 and 1,000,000 unique integer IDs and inserts the student registration number as a personal found key. The test key 192411048 is used as the required absent key. Each algorithm is checked for both found and not-found outcomes.

The benchmark uses the same underlying generated dataset for the three approaches. Binary search receives a sorted copy and its sorting time is measured separately. Hash lookup receives an index built from the unsorted records, and index construction time is measured separately. The program prints student identity, timestamp, runtime environment and results.

Correctness checks implemented: linear search returns a valid index for the student key and -1 for the absent key; recursive binary search does the same on sorted records; hash lookup returns the key when present and None when absent.

Evidence still required: capture genuine full terminal screenshots on your own computer after running the commands in README.md. The submission rules require name or registration number and system date/time to be visible. Do not submit the report as fully complete until those screenshots are embedded and the repository link is added.

Task 9 – Record Experimental Performance

The benchmark recorded the time for two queries (student registration number found, registration number + 1 absent), comparison counts where meaningful, and preprocessing time. Times below are measurements from one run and will vary across machines. Hash membership operations are not directly comparable to array key comparisons.

Measured results

n	Algorithm	Preprocess (s)	2-query search (s)	Comparisons / note
1,000	Linear search	0.000000	0.000047	1001
1,000	Recursive binary search	0.000063	0.000008	20
1,000	Hash index lookup	0.000015	0.000001	not directly comparable
10,000	Linear search	0.000000	0.000528	10001
10,000	Recursive binary search	0.001089	0.000008	28
10,000	Hash index lookup	0.000361	0.000002	not directly comparable
100,000	Linear search	0.000000	0.005631	100001
100,000	Recursive binary search	0.015723	0.000018	34
100,000	Hash index lookup	0.020795	0.000005	not directly comparable
1,000,000	Linear search	0.000000	0.228387	1000001
1,000,000	Recursive binary search	0.334780	0.000028	40
1,000,000	Hash index lookup	0.229686	0.000003	not directly comparable

The CSV results file is the source of the report plots. The graphs use logarithmic axes because the measured values span several orders of magnitude.

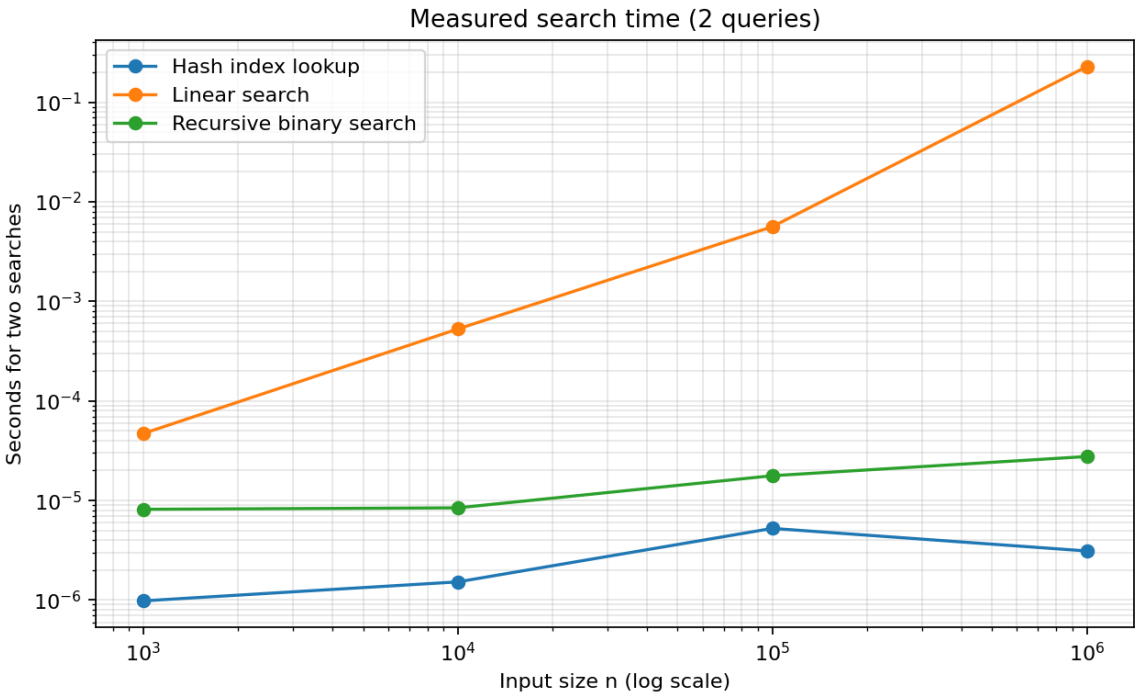


Figure 9.1: Measured search time for two queries, by input size. Preprocessing is excluded from this graph.

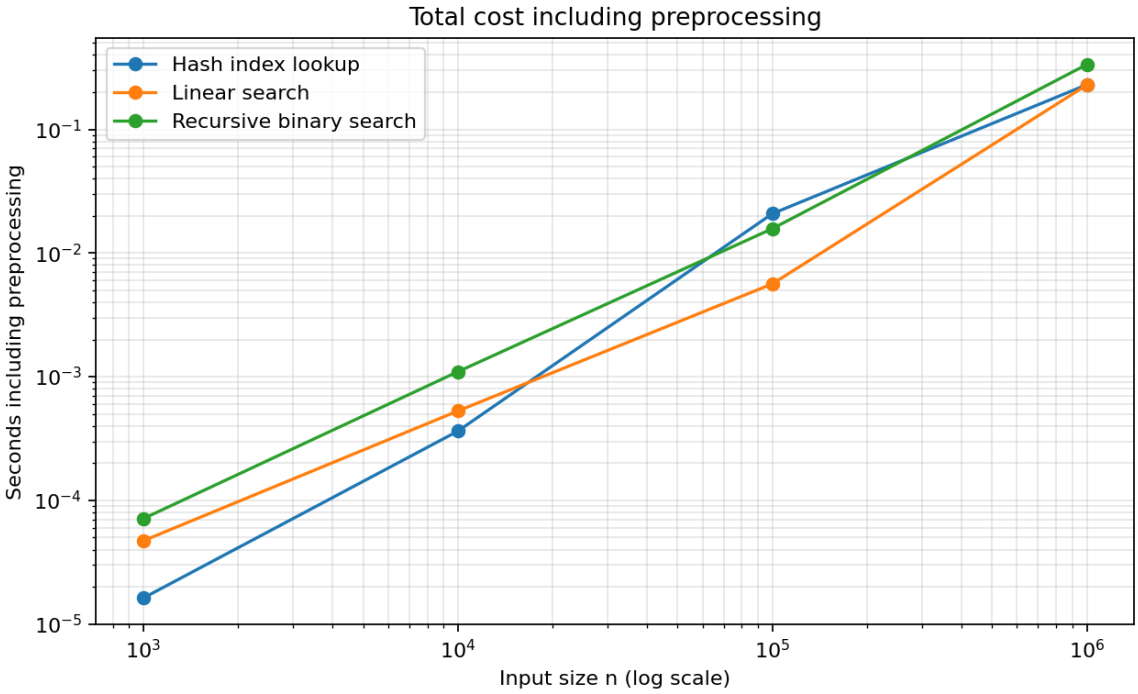


Figure 9.2: Measured total cost including sorting or hash-index construction.

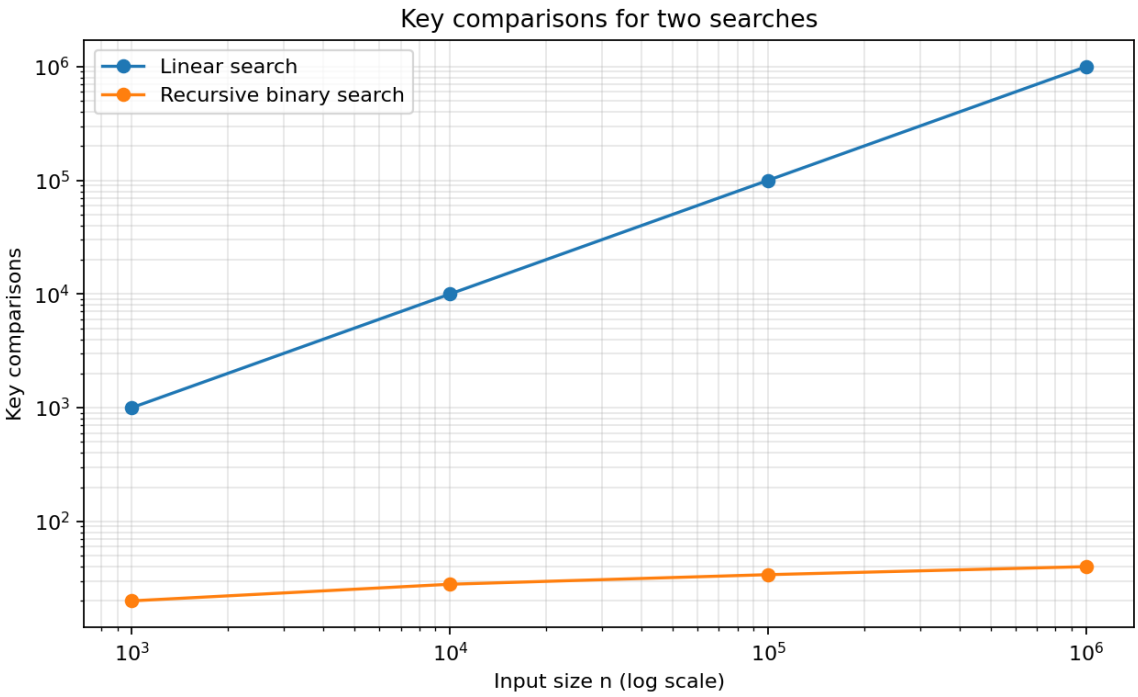


Figure 9.3: Key-operation comparison counts. Hash operations are omitted because their count is not directly comparable.

Task 10 – Compare Theory with Experimental Behaviour

The measured linear-search comparison count rises from 1,001 across the two queries at n = 1,000 to 1,000,001 at n = 1,000,000. This is consistent with a missing-key scan of n records plus a found-key scan that reaches the inserted key near the front of the generated list. Runtime rises substantially at the largest size, which agrees with linear growth.

Recursive binary search records 20, 28, 34 and 40 midpoint visits across two queries for the four sizes. The growth is slow and consistent with logarithmic behaviour. Search-only time is small, but total cost includes sorting. At one million records, sorting dominates the two-query search time in this one-shot benchmark.

Hash lookup has very small measured query time, but index construction is a material one-time cost. At one million records, the build takes around a fraction of a second in this environment, while the two membership checks take only a few microseconds. Timing is affected by Python overhead, memory allocation, CPU scheduling, cache effects and timer resolution. The experiment supports the asymptotic trend but does not establish universal timings.

Figure 10.1: Search-only and total-cost plots in Task 9 provide the visual comparison between measured behaviour and theoretical growth. For a formal theory-versus-experiment visual, the CSV is included so the graph can be regenerated after rerunning the benchmark on the submission computer.

Task 11 – Recommend and Defend the Final Design

For a system with repeated lookups and records that do not change frequently, recommend a hash index, provided memory for the additional index is available and expected hash behaviour is acceptable. Index construction is expected O(n), while each lookup is expected O(1). The cost is paid once and can be amortized over many queries. The benchmark shows the hash index has low query latency; it also records the build cost separately so the trade-off is visible.

Recursive binary search is a reasonable alternative when ordered traversal is useful, the records are already sorted, or memory and indexing constraints favour an array. It provides deterministic O(log n) search after sorting, but sorting an unsorted dataset costs O(n log n). Linear search is easiest to implement and needs no preprocessing, but it scales poorly for large collections and frequent absent-key queries.

The recommendation is based on the workload, not just Big-O notation. For a small number of queries on a newly loaded dataset, preprocessing may outweigh the benefit of an index. For repeated queries against a mostly static dataset, hashing is more suitable. The hash set stores only keys in this demonstration; a production system would map each registration number to the corresponding complete student record and would need persistence, concurrency control and collision-resistant implementation details.

Task 12 – Day-4 Design Challenge

A. Impact analysis

The revised workload specifies 50,000 searches per hour against 1,000,000 records, while new records are added only once per day. Search volume is now explicit and high relative to update frequency. The original assumption that preprocessing is a one-time cost becomes more favourable because the index can serve many searches before the next daily update. The hourly rate averages about 13.9 searches per second, though real traffic may arrive in bursts.

B. Evaluation of the original design

If the original recommendation is hash indexing, it remains suitable. The index is built once and used for repeated lookups. Binary search can also serve the workload, but it needs sorted data and each query performs logarithmic work; linear search is not appropriate at this scale because its work grows directly with the record count. The experiment measured 50,000 successful queries on one million records: hash lookup took 0.021235 seconds after index construction of 0.180668 seconds, while recursive binary search took 0.262194 seconds over the same keys after sorting. These are local single-run results, not production capacity guarantees.

C. Adaptation or redesign

Modify the original design into a persistent hash index keyed by registration number. Load or rebuild it after the daily batch update, then route incoming lookups to the index. Expected build time is O(n), expected lookup time O(1), and expected total work for q searches is O(n + q). Space remains O(n). If daily updates are incremental, insert the new records into the index instead of rebuilding it, with expected O(1) insertion per record. Keep a database or durable source of truth behind the index so a restart does not lose student records.

D. Evidence-based defence

The one-million-record benchmark found all 50,000 keys in both hash and binary approaches. The measured hash query loop was faster in this environment, and the build cost was small relative to serving a sustained stream of requests. With 50,000 queries each hour and daily updates, the expected O(n) build cost is amortized across many queries. The design must still be checked under realistic concurrency, memory limits, unsuccessful lookups, and update/restart conditions before production use.

```
Reg No: 192411047 | Name: P SAI DEEPAK
Timestamp: 2026-10-16T08:13:10+00:00
n=1000000; queries=50000; hash_build_seconds=0.180668; hash_query_seconds=0.021235; binary_50000_seconds=0.262194; hash_four
```

Figure 12.1: Benchmark summary produced by the included code. Re-run on the target computer to produce matching local output and take the required genuine terminal screenshot.

Appendix – Reproduction Instructions and Submission Checklist

From the repository root, run: python src/benchmark.py. Then run: python src/make_graphs.py. The first command recreates results/results.csv and results/challenge_summary.txt. The second regenerates graphs from the CSV. The generator seed is 1047. Python 3.10+ is recommended.

Before submitting: (1) replace the repository placeholder on page 1 with the actual GitHub URL; (2) create at least five meaningful commits over the week as required; (3) take genuine full terminal screenshots with the system date/time and student identity visible; (4) embed the screenshots in this PDF and put copies in screenshots/; (5) verify whether the instructor assigned hash indexing or a different third strategy for your roll-number group; (6) rerun the benchmark and confirm the report tables match results.csv; (7) upload the PDF, code ZIP and GitHub link to Classroom.

Integrity note: benchmark numbers in this draft are actual measurements from the included code in the generation environment, but your local run may differ. Do not claim that generated text excerpts are screenshots. Do not submit mismatched screenshots or results.